Local Agentic Retail Interview Case

Senior AI Engineer and AI Architect

Preparation window 7 calendar days from receipt. Submit a working demo and an architecture and results presentation.

The case

Build a local retail returns and exchanges desk. A customer describes a request; your agents inspect the order, apply policy, check replacement stock, clarify missing details, propose a remedy, obtain explicit confirmation, and execute the approved action. The interface must show what happened and recover clearly when a tool fails.

Required implementation

- Use at least 2 cooperating LLM agent roles with distinct responsibilities, such as customer resolution and retail operations. Show tool calls, decisions, handoffs, and outcomes. Additional agents earn no automatic credit.
- Run all inference, retrieval, business tools, and storage locally. After dependencies and models are downloaded, the entire demo must work with internet access disabled. AI coding assistants are encouraged during development; disclose the tools used.
- Build a focused web UI with request entry, remedy details, confirmation, execution status, and an understandable audit trail. Keep money calculations, authorization checks, and transaction invariants in deterministic code.
- Implement the shared JSON adapter contract and resettable state in the accompanying candidate_assets folder. Use its fixtures unchanged for comparable tests. The assets supply data and a contract; you implement the application and simulated business tools.

Business scope

Support refunds and same-product, same-price size exchanges only. Eligibility requires a delivered, unopened, non-final-sale item within 30 calendar days of delivery, inclusive. Use the fixture date, not the computer clock. Refunded or exchanged quantities cannot be processed again.

Refund the original net amount paid in integer USD cents. Exclude tax, shipping, fees, real payment services, and price-difference collection. Returned units enter quarantine, not sellable inventory. Exchanges reduce replacement stock. Bind approval to the proposed action; recheck eligibility and stock before committing. Repeated requests must not duplicate actions.

Hardware and expenses

Use your own available equipment. No specific GPU or hardware purchase is required. We provide no machine and do not reimburse hardware, software, or development expenses. Choose models and frameworks that run on your equipment. Disclose CPU, GPU and count, RAM, VRAM or unified memory, OS, driver, model revision, quantization, inference engine, and power settings.

Submission and presentation

- Submit source code or a repository link, dependency and model versions, setup and run instructions, the JSON adapter, tests, and a reset command. Include raw benchmark results and known limitations. Exclude secrets and unnecessary model binaries.
- Prepare a presentation of at most 10 slides. Explain agent boundaries, data and tool flows, model and inference-engine choices, failure handling, evaluation, baseline versus final results, tradeoffs, and remaining limitations. Separate measured results from proposed improvements.
- Plan for a 5-minute live demo, a 10-minute presentation, and 15 minutes of questions and a small live change. Demonstrate a successful transaction and failure recovery. You must explain and modify the submitted code, including AI-generated portions.

Evaluation and evidence

Both candidates receive the same rules, public fixtures, and technical contract. Evaluation may use unseen fixture variants within that contract. Support ambiguous requests, policy rejection, stock changing before confirmation, duplicate requests, and a timeout after an action has committed. Never invent a successful transaction.

Run every public scenario 3 times from a clean initial state. Compare a reasonable working baseline with your final version on the same device and test set. Record complete-task success, p50 and p95 successful-task latency, failures and timeouts, model token throughput, peak RAM and accelerator memory, and cold startup separately from warm execution. Declare seeds, concurrency, caches, and timeouts; provide reproducible commands and per-run records.

Raw speed and memory use on different machines will not be ranked directly. Optimization is scored on experimental rigor, quality preservation, justified changes, and reproducibility. A larger percentage speedup from a deliberately weak baseline earns no advantage. Unsupported metrics must be marked unavailable.

Criterion	Weight	What we assess
Business correctness	25%	Correct eligibility, refund amounts, inventory and final transaction state.
Agent reliability	20%	Effective tool use, clarification, approval, recovery and duplicate prevention.
Architecture and code	20%	Clear boundaries, maintainability, meaningful tests, reproducible setup and technical defense.
Local optimization	15%	Credible baseline, controlled measurements, resource awareness and preserved quality.
UI and UX	10%	Clear proposals, usable confirmation, visible progress and actionable errors.
Presentation and evidence	10%	Traceable results, justified choices, honest limitations and understanding of the implementation.

Automated checks assess functional results; architecture, code, UI and presentation also receive structured human review. Local execution and meaningful agent use are mandatory requirements. Share clarification questions early so the same answers can be provided to both candidates.